

AppSec Agents Guide

Use this as a terse checklist when writing, reviewing, or fixing code. Keep data and code separate. Prefer allowlists over blocklists. Prefer secure platform defaults over custom security code.

Agentic Workflow

- Keep this file at `agents/APPSEC-AGENTS-GUIDE.md` as the repo's AppSec source of truth.
- Codex and similar agents: reference this file from the repo-root `AGENTS.md`, or place the guidance directly in `AGENTS.md` for always-on review rules.
- Claude Code: add `@agents/APPSEC-AGENTS-GUIDE.md` to the repo's `CLAUDE.md`.
- Aider: load it with `/read agents/APPSEC-AGENTS-GUIDE.md`, `aider --read agents/APPSEC-AGENTS-GUIDE.md`, or `.aider.conf.yml`.
- Ask the agent to consult this guide before security-sensitive edits and before final review.

Blocklists, Escaping, Encoding

- Blocklists fail because dangerous syntax is context-specific, encodings are ambiguous, parsers disagree, and attackers can find forms the list did not predict.
- Escaping tries to neutralize selected characters after mixing data into code; it is brittle when the output context changes.
- Encoding converts data into the safe representation for its exact destination before interpretation: HTML text, HTML attribute, JS string, CSS, URL, SQL parameter, shell arg.
- Validate input with strict allowlists for business rules, then encode or parameterize at the sink.

Bad JSP blocklist:

```
String name = request.getParameter("name");
name = name.replace("<script>", "").replace("</script>", "");
out.println("<p>Hello " + name + "</p>");
```

Good JSP/servlet encoding:

```
String name = request.getParameter("name");
out.println("<p>Hello " + org.owasp.encoder.Encode.forHtml(name) + "</p>");
```

Triage and Scanning

- Start with the [Application Security site](#) and the [Vulnerability Triage Field Guide](#).
- Scan code in the IDE with Wiz IDE plugins before commit and while fixing findings.
- Follow the [Wiz Dev Quick Start Guide](#).
- Set up StackHawk for local/dev DAST scanning and run it before release.
- Follow the [StackHawk Dev Quick Start](#).
- Use scanner findings to find the root cause; fix the vulnerable pattern, not only the reported line.

Common Fixes

CWE-89: SQL Injection

- Never build SQL with string concatenation, interpolation, or template strings.
- Use prepared statements, parameterized queries, or safe ORM/query-builder APIs.
- If dynamic identifiers are unavoidable, map user choices to a strict allowlist of known table/column names.

CWE-90, CWE-643, CWE-943: LDAP, XPath, and NoSQL Injection

- Do not build LDAP filters, XPath expressions, or NoSQL queries from raw user-controlled strings/JSON.
- Use safe framework/driver APIs. Encode for LDAP DN/filter contexts and bind or parameterize XPath where supported.
- For NoSQL, reject client-controlled operators such as `$where`, `$regex`, `$expr`, `$ne`, and `$gt` unless explicitly required and allowlisted.
- Do not pass raw JSON fragments, raw expressions, or `eval`-like query features to the database.
- Run service accounts with least privilege so injection cannot read or modify everything.

CWE-78: Command Injection

- Avoid shell execution for app logic. Prefer language/platform APIs.
- If execution is required, pass executable and args as separate values; do not invoke a shell or build one command string.
- Strictly allowlist user-selectable arguments; reject metacharacters by design, not by fragile blocklists.

CWE-79: Cross-Site Scripting (XSS)

- Do not build HTML/JS/CSS with raw user-controlled strings.
- Encode for the exact output context: HTML text, attribute, URL, JS string, CSS.
- Prefer DOM APIs that create text safely: `textContent`, `innerText`, `createTextNode`.
- Avoid `innerHTML`, unsafe template rendering, inline event handlers, and `javascript:` URLs.
- Sanitize only with a maintained HTML sanitizer when rich HTML is truly required.

CWE-1336: Server-Side Template Injection

- Never concatenate user input into template source code.
- Keep templates developer-authored; pass user data only as template variables.
- If users must author templates, use a sandboxed template engine with dangerous functions, imports, file access, and reflection disabled.
- Do not expose secrets, request objects, application config, shell helpers, or unrestricted object graphs to templates.

Headers, CORS, CSP

- Set a restrictive Content Security Policy. Start with `default-src 'self'; object-src 'none'; base-uri 'self'; frame-ancestors 'none'`.
- Use nonces or hashes for scripts. Avoid `unsafe-inline` and broad script domains.

- Set CORS to exact trusted origins only. Do not use `Access-Control-Allow-Origin: *` with credentials.
- Set `Strict-Transport-Security`, `X-Content-Type-Options: nosniff`, `Referrer-Policy`, and `Permissions-Policy`.
- Use `HttpOnly`, `Secure`, and `SameSite` cookies for session tokens.
- Prefer CSP `frame-ancestors` over legacy clickjacking headers; add `X-Frame-Options` only for legacy support.

CWE-352: Cross-Site Request Forgery (CSRF)

- Protect every state-changing request that relies on browser credentials.
- Use the framework's CSRF protection or cryptographically strong synchronizer/signed double-submit tokens.
- Do not put CSRF tokens in URLs or rely on a token cookie by itself.
- Use `SameSite=Lax` or `Strict` cookies and verify `Origin/Referer` as defense in depth.
- Do not change server state with `GET`. Remember that XSS can bypass CSRF defenses.

JWT and Token Misconfiguration

- Use a vetted JWT library; do not hand-roll token parsing or signature checks.
- Lock accepted algorithms and reject `none`, algorithm confusion, and unexpected key types.
- Always validate signature, `exp`, `nbf`, `iss`, `aud`, subject, tenant, scopes/roles, and intended token use.
- Keep access tokens short-lived; support revocation, key rotation, and a strict `kid` allowlist.
- Do not store sensitive data in readable JWT claims unless the token is encrypted.
- For browser apps, prefer server-side sessions when revocation matters; otherwise store tokens with strong cookie/session protections.

TLS and Certificate Validation

- Do not disable TLS verification. Fix trust stores and certificates instead.
- Follow: [How to fix SSL/TLS errors the right way](#).
- Follow: [Testing for weak SSL ciphers and misconfigurations](#).
- Validate hostnames and certificate chains. Never use `verify=False`, permissive trust managers, or "accept all" hostname verifiers.

TLS Configuration

Web applications must:

- Provide scan evidence that weak TLS 1.2 ciphers are deprecated.
- Use TLS 1.2 or greater.
- Reject SSL, TLS 1.0, and TLS 1.1.
- Reject NULL, RC2, RC4, DES, IDEA, and 3DES.
- Reject CBC cipher modes.
- Use certificates with at least 2048-bit RSA or 256-bit ECC keys.
- Use SHA-256 or stronger certificate signatures; do not use SHA-1.
- Enable HSTS.
- Require PFS with ECDHE key exchange.

Web applications should:

- Prefer TLS 1.3.
- Prefer CCM, GCM, or CHACHA20_POLY1305 cipher modes.
- Enable OCSP stapling.
- Automate certificate expiration monitoring and renewal.
- Enforce server-preferred cipher suite order.
- Accept modern suites such as:
 - `TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384`
 - `TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256`
 - `TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384`
 - `TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256`
 - `TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256`
 - `TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256`

Cryptography

- Avoid AES-CBC for new code. Use authenticated encryption: AES-GCM, ChaCha20-Poly1305, or vetted platform AEAD APIs.
- Never hard-code cryptographic keys. Load keys from an approved secret/key store and support rotation.
- Use random nonces/IVs as required by the algorithm. Never reuse AEAD nonces with the same key.

Email and Telephone Parsing

- Do not parse emails or phone numbers with homegrown regex.
- Use platform parsers or maintained RFC/standard-aware libraries, such as `email.utils`, `mailaddress`, `libphonenumber`, or equivalent.
- Treat validation as normalization plus business rules, not proof that an address or number exists.

CWE-1321: Prototype Pollution

- Reject `__proto__`, `prototype`, and `constructor` in user-controlled keys, including nested keys.
- Validate object shapes before merges, path setters, query parsing, or `Object.assign`.
- Prefer `Map`, `Set`, or `Object.create(null)` for key-value data.
- In Node.js, consider `--disable-proto=delete` or `--disable-proto=throw` as defense in depth.

CWE-918: SSRF

- Do not fetch arbitrary user-provided URLs from the server.
- Allowlist trusted schemes, hosts, and ports before making outbound requests.
- Block private IP ranges, localhost, link-local, metadata services, and non-HTTP handlers by default.
- Resolve DNS and re-check the final IP after redirects.

CWE-611: XML External Entity (XXE) and DTD Resolution

- Do not parse untrusted XML with default parser settings.
- Disable DTD/DOCTYPE processing for untrusted XML whenever possible.

- Disable external general entities, external parameter entities, external DTD loading, XInclude, and remote schema fetching.
- If DTDs are required, use only pinned local DTDs with a deny-by-default entity resolver/XML resolver.
- Prevent parser file/network access and set strict expansion, depth, and size limits to avoid entity expansion DoS.

CWE-502: Insecure Deserialization

- Do not deserialize untrusted native objects, such as Java serialization, Python pickle, PHP `unserialize`, or .NET binary serializers.
- Prefer simple data formats such as JSON with schema validation and explicit DTOs.
- If deserialization is unavoidable, enforce strict type allowlists and reject polymorphic/gadget-prone types.
- Sign or authenticate serialized data, but do not treat signatures as permission to deserialize attacker-controlled object graphs.
- Run deserialization with least privilege, size/depth limits, and no network/file side effects.

CWE-35: Path Traversal

- Never build filesystem paths with string concatenation.
- Use OS path APIs, canonicalize/resolve paths, then verify the result remains inside the intended base directory.
- Prefer server-side IDs mapped to allowed files instead of user-supplied paths.

CWE-601: Open Redirect

- Do not redirect to arbitrary user-controlled URLs.
- Use relative redirects or map return targets to a strict allowlist of trusted domains.
- Reject mixed-scheme, encoded, nested, or userinfo-style redirect tricks.

CWE-798 and CWE-321: Hard-Coded Secrets or Keys

- Do not commit passwords, tokens, connection strings, private keys, or crypto keys.
- Move secrets to environment-backed configuration or approved vault/key-management systems.
- Rotate exposed secrets immediately; removing them from code is not enough.

CWE-489 and CWE-215: Debug Code and Sensitive Debug Output

- Disable debug routes, verbose errors, stack traces, test consoles, and diagnostic dumps in production.
- If debug access is unavoidable, require strong authentication and authorization.
- Never log or display secrets, tokens, credentials, full environment dumps, or sensitive paths.

CWE-121: Stack-Based Buffer Overflow

- Avoid unsafe C/C++ buffer/string APIs and unchecked pointer arithmetic.
- Validate lengths before writes; prefer bounds-checked APIs and memory-safe languages.
- Compile with hardening such as stack canaries, ASLR, DEP/NX, and FORTIFY where available.

CWE-295: Improper Certificate Validation

- Always validate certificates and hostnames.
- Fix local trust problems by installing the correct enterprise/root certificates; do not bypass validation.
- See: [How to fix SSL/TLS errors the right way](#).

CWE-200: Sensitive Information Exposure

- Do not expose internals such as file paths, stack traces, environment variables, process names, keys, or detailed service metadata.
- Return generic user-facing errors; keep detailed diagnostics in protected logs.
- Mask secrets in logs and telemetry.

CWE-942: Permissive Cross-Domain Policy

- Configure CORS with exact trusted origins, methods, and headers.
- Do not use wildcard origins for authenticated APIs.
- Use CSP to restrict script, style, connect, image, frame, and form destinations.

Agent Review Prompts

- Is user input ever interpreted as SQL, LDAP, XPath, NoSQL, shell, template code, HTML, XML, URL, file path, object key, redirect target, serialized object, JWT claim, or crypto material?
- Did the fix separate data from code using platform APIs?
- Are allowlists small, explicit, and enforced before use?
- Are TLS, cert validation, headers, cookies, and CORS secure in production defaults?
- Are secrets, debug outputs, and sensitive errors removed from source and runtime responses?